

HW 1

1. Write $51/8$ in fixed-point and floating-point arithmetic (using 32 bits)

$$\frac{51}{8} = 6 + \frac{3}{8} = 2^5 + 2^9 + 2^1 + 2^0 + 2^{-2} + 2^{-3} = (110011.011)_2$$

$$\frac{51}{8} = (1.10011011)_2 \times 2^5$$

fixed: 0 ; 00...0110011 ; 01100...00

floating: 0 ; 00...0101 ; 1001101100...00

2. Instead of assigning one bit to record the sign (pos or neg), this new machine uses a 2's complement scheme, in which positive integers $0 \leq x \leq 2^n - 1$ are stored like normal but negative integers $-y$ for $1 \leq y \leq 2^{n-1}$ are stored as the binary repr. $2^n - y$

a) Write the 2's complement repr. of -63

$$-63 = -(2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0) = -(111111)_2$$

$$\begin{array}{r} 2's: 100...000000 \\ - \quad 111111 \\ \hline 11...1000001 \end{array} \quad \rightarrow \quad \boxed{11...11000001} \quad 32\text{-bit}$$

b) Add the 2's complement repr. of 63 and -63

$$63 = (111111)_2$$

$$\begin{array}{r} 63 \\ + (-63) \\ \Rightarrow \begin{array}{r} 00...00111111 \\ + 11...11000001 \\ \hline 00...00000000 \end{array} \end{array}$$

throw away leading overflow bit

3. Prove the following: in IEEE floating point arithmetic,

$$\text{round}(x) = \frac{x}{1+\delta}$$

for some $|\delta| \leq \epsilon_{\text{mch}}$, where ϵ_{mch} is machine epsilon

From class, we discussed the theorem

$$\text{round}(x) = x(1+\delta_1)$$

where $|\delta_1| \leq \epsilon_{\text{mch}}$. Now, we must choose a δ such that $1+\delta = \frac{1}{1+\delta_1}$

$$\Rightarrow \delta = \frac{1}{1+\delta_1} - 1 = \frac{1 - (1+\delta_1)}{1+\delta_1} = \frac{-\delta_1}{1+\delta_1}$$

We can verify that our δ is valid with $\left| \frac{-\delta_1}{1+\delta_1} \right| \leq \epsilon_{\text{mch}}$

$$\Rightarrow \left| \frac{\delta_1}{1+\delta_1} \right| \leq \frac{|\delta_1|}{1-|\delta_1|} \leq \frac{u}{1-u} \leq 2u = \epsilon_{\text{mch}}$$

where u is the unit roundoff $u = 2^{-n}$ and $|\delta_1| \leq u$. Since we have defined a δ such that $\frac{1}{1+\delta} = 1+\delta_1$,

$$\text{round}(x) = \frac{x}{1+\delta}, \quad |\delta| \leq \epsilon_{\text{mch}}$$

4. $x^2 - 56x + 2$

Analysis: The standard quadratic formula is more accurate with respect to the first true root, $r_1 = 55.9643$, yet the alternative formula is more accurate with respect to the second true root, $r_2 = 0.035737$.

$x^2 + 56x + 2$

Analysis: The alternative formula is more accurate with respect to the first true root, $r_1 = -0.035737$, yet the standard quadratic formula is more accurate with respect to the second true root, $r_2 = -55.9643$. It seems my answer "flipped" when changing the sign $\pm 56x$.

Hypothesis: The difference in performance may be due to changing the sign for the 'b' term. Since the alternative formula divides by the expression with the 'b' term, perhaps it is more accurate in calculating the small root when the divisor magnitude is large. I suspect that this division of the 'b' expression preserves more significant digits than it being in the numerator for the standard formula.

Code is on next page ↓↓

James Dinh

```

function [r1, r2] = trueRoots(a, b, c)
    sqrtTerm = sqrt(b^2 - (4*a*c));
    r1 = (-b + sqrtTerm) / (2*a);
    r2 = (-b - sqrtTerm) / (2*a);
end

function [r1, r2] = quadRoots(a, b, c)
    sqrtTerm = sqrt(b^2 - (4*a*c));
    z = round(sqrtTerm, 5, 'significant');
    r1 = (-b + z) / (2*a);
    r2 = (-b - z) / (2*a);
end

function [r1, r2] = altRoots(a, b, c)
    sqrtTerm = sqrt(b^2 - (4 * a * c));
    z = round(sqrtTerm, 5, 'significant');
    r1 = (2*c) / (-b - z);
    r2 = (2*c) / (-b + z);
end

poly = [1, 56, 2];

[r1, r2] = trueRoots(poly(1), poly(2), poly(3));
disp(['True roots: r1 = ', num2str(r1), ', r2 = ', num2str(r2)]);

[r1, r2] = quadRoots(poly(1), poly(2), poly(3));
disp(['Roots using Standard Quadratic Formula: r1 = ', num2str(r1), ', r2 = ', num2str(r2)]);

[r1, r2] = altRoots(poly(1), poly(2), poly(3));
disp(['Roots using Alternative Formula: r1 = ', num2str(r1), ', r2 = ', num2str(r2)]);

True roots: r1 = -0.035737, r2 = -55.9643
Roots using Standard Quadratic Formula: r1 = -0.0355, r2 = -55.9645
Roots using Alternative Formula: r1 = -0.035737, r2 = -56.338

```

Published with MATLAB® R2025b